

NCollection ERP — Security Architecture

Version: 1.0 **Date:** July 16, 2026 **Classification:** Internal — Enterprise Engineering Reference

Purpose: The authoritative security design for the NCollection ERP Platform. The platform hosts many companies' financial, HR, and commercial data on shared infrastructure, with a custom SaaS layer on top of Odoo — meaning it inherits Odoo's attack surface **plus** every surface the platform layer adds (provisioning, licensing, billing, public checkout, cross-DB sync). This document defines the threat model, the layered defense design, and the per-phase security checklists that PRs and phase gates are reviewed against.

Companion: [ARCHITECTURE_DATA_PLATFORM.md](#) (data backbone) · [DELIVERABLE_1_SYSTEM_DESIGN.md](#) (task plan)

Table of Contents

1. [Security Principles](#)
 2. [Threat Model](#)
 3. [The Five Defense Layers](#)
 4. [License Enforcement — Defense in Depth](#)
 5. [Tenant Isolation Guarantees & Continuous Verification](#)
 6. [Authentication & Session Security](#)
 7. [Secrets Management](#)
 8. [Data Protection: Encryption & Backups](#)
 9. [Compliance: UAE PDPL & Beyond](#)
 10. [Security Operations](#)
 11. [Platform-Layer Specific Risks](#)
 12. [Per-Phase Security Checklists](#)
-

1. Security Principles

1. **UI hiding is never security.** Every restriction visible in the UI must have a matching enforcement at the ORM/RPC layer. A hidden menu with an accessible model is a vulnerability, not a feature. (Rule 7 in the master plan.)
2. **The tenant boundary is sacred.** No feature, optimization, or convenience may create a code path where one tenant's request can touch another tenant's data. Cross-tenant isolation is verified by automated tests on every PR (P1-T20), not by intention.
3. **Assume hostile tenants.** Any subscriber can be a curious power user, a disgruntled employee with valid credentials, or an attacker with a stolen password. Isolation and licensing must hold against *authenticated* adversaries, not just anonymous ones.

4. **OCA before custom for security code.** Custom security code (auth, crypto, session handling) is where teams create their own vulnerabilities. Battle-tested OCA modules (`auth_brute_force` , `auth_session_timeout` , `auditlog`) are preferred; custom security code requires a second reviewer.
5. **Least privilege everywhere.** Service accounts, satellite containers, DB roles, deploy users, API scopes — each gets exactly the access its one job requires (see [ARCHITECTURE_DATA_PLATFORM.md §10.4](#)).
6. **Secure by default at provisioning.** A freshly provisioned tenant must be born hardened: strong forced-reset admin password, licensed modules only, no debug access, hardened session settings. Security cannot depend on post-setup manual steps.
7. **Evidence, not assertion.** Every security claim in a phase gate is backed by a linked test run, scan report, or drill log.

2. Threat Model

2.1 Assets

Asset	Sensitivity
Tenant business data (invoices, payroll, customers, pricing)	Critical — breach is existential
Platform admin DB (all tenants' metadata, billing, credentials)	Critical — compromise = full-platform compromise
Authentication credentials & sessions	Critical
Backups (contain everything above)	Critical — often the softest copy of the hardest target
Payment flows (Stripe/PayTabs webhooks, invoices)	High
Platform availability	High — tenants run their business on it
Source code & CI pipeline	High — supply-chain vector into every tenant

2.2 Threat Actors & Primary Vectors

Actor	Capability	Primary vectors	Key mitigations
Anonymous internet	Scanning, credential stuffing, exploit kits	Login brute force, exposed DB manager, unpatched CVEs, checkout abuse	Edge rate limits (P1-T03), <code>auth_brute_force</code> (P1-T19), <code>/web/database/*</code> blocked, patch cadence (§10), reCAPTCHA (P2-T16)
Authenticated tenant user (curious/malicious)	Valid login, RPC access, browser dev tools	License bypass via RPC, privilege escalation through <code>implied_ids</code> , debug mode, IDOR on portal	ORM license enforcement (P1-T10), role matrix audit (P1-T08), debug blocking (P1-T11), record rules + IDOR tests (P6-T02)
Tenant admin (Owner)	Full control of own workspace	Attacks on the platform layer from inside a tenant, resource abuse	Two-layer separation (no platform models in tenant DBs), quotas, workspace-scoped settings only (P1-T12)

Compromised tenant account	Everything its role allows	Data exfiltration, fraud	Auth log (P1-T19), audit trail (P8-T05), 2FA (P10-T04), session timeout
Malicious insider / stolen dev credentials	Repo, CI, servers	Supply chain, direct DB access	Branch protection + reviews, SSH keys + fail2ban (P2-T08), secrets separation (§7), audit
Third-party dependency compromise	Code execution in-process	Malicious OCA/pip package update	Pinned versions (<code>repos.yml</code> , P1-T04), <code>pip-audit</code> / <code>trivy</code> in CI (P1-T05), review of dependency bumps

2.3 Attack Surfaces Introduced by the Platform Layer (not present in vanilla Odoo)

This is the crucial insight: **the platform layer adds surfaces Odoo never had**, and they get dedicated controls (§11):

1. **Provisioning engine** — creates databases and admin users programmatically; a flaw here mints attacker-controlled tenants or overwrites existing ones.
 2. **Config sync channel (P2-T03)** — a privileged cross-DB write path; its service account is one of the most powerful credentials in the system.
 3. **Public checkout (P2-T16)** — unauthenticated form that triggers resource creation (DBs!) — DoS and abuse magnet.
 4. **License enforcement (P1-T10)** — enforcement code itself becomes a target; bugs here have direct revenue impact.
 5. **Payment webhooks (P2-T13/P6-T01)** — forged webhook = free subscriptions or falsely-paid invoices.
 6. **Admin DB** — a single database whose compromise affects every tenant.
-

3. The Five Defense Layers

— LAYER 1: EDGE (Nginx) —	
TLS 1.2+ only · HSTS · security headers (CSP, X-Frame-Options, nosniff, Referrer-Policy) · rate limits on /web/login & checkout · /web/database/* blocked · request size limits · gzip w/o BREACH-sensitive paths	P1-T03
— LAYER 2: APPLICATION (Odoo + OCA) —	
auth_brute_force lockouts · session timeout · secure cookies (Secure, HttpOnly, SameSite=Lax) · CSRF (Odoo built-in – never bypass) · auth event log · list_db=False · debug blocked	P1-T19
— LAYER 3: AUTHORIZATION (ORM) —	
8-role matrix (implied_ids audited) · ir.model.access ACLs · ir.rule record rules · LICENSE ENFORCEMENT on unlicensed models (menu hiding is Layer 3's UI shadow, never its substance)	P1-T08/T09/T10/T11
— LAYER 4: DATA (PostgreSQL + storage) —	
DB never exposed publicly · db_filter=%d\$ scoping · least-priv DB roles · encrypted backups & WAL archive · per-tenant filestore separation · disk encryption on VPS volumes	P2-T04/05/08/09
— LAYER 5: OPERATIONS —	
UFW 22/80/443 only · SSH keys + fail2ban · unattended security updates · pinned deps + CI scanning · secrets lifecycle · audit trail · monitoring & alerting · incident runbook	P2-T07/08/10 ff.

Review rule: a PR touching any layer states which layer(s) it affects and which checklist items (§12) it satisfies. Security-sensitive PRs (Layers 2–4) require review against this document.

4. License Enforcement — Defense in Depth

The subscription-based module licensing is the product's revenue mechanism **and** a security control. It is built as three concentric rings:

Ring 1 — Menu visibility (UX) — P1-T09

`ir.ui.menu._visible_menu_ids()` override removes unlicensed modules' menus. Purpose: clean UX and upsell surface. **Provides zero security.**

Ring 2 — ORM enforcement (the real control) — P1-T10

- The tenant's `ncollection.workspace.config` maps the licensed module set → allowed model namespaces.
- An `AbstractModel` mixin / `check_access_rights` override (plus generated `ir.rule` where appropriate) denies **read/write/create/unlink** on models belonging to unlicensed modules, for all non-system users.
- Enforcement returns a branded "not included in your plan" response where the UI can render it (upsell moment), and a plain `AccessError` over raw RPC.
- Performance budget: < 5 ms per request; the allowed-set is cached and invalidated on config sync (P2-T03).

Ring 3 — Non-installation (blast-radius reduction) — P2-T01

Provisioning installs **only** the plan's modules. What isn't installed cannot be attacked, licensed or not. Plan upgrades install additions via the sync/provisioning channel — never by the tenant.

Verification (continuous, not one-off): the E2E suite (P1-T20) logs into a Starter tenant and (a) asserts unlicensed menus are absent, (b) navigates directly to an unlicensed action URL → expects denial, (c) fires XML-RPC `search_read` against an unlicensed model → expects `AccessError`, (d) repeats all three against an Enterprise tenant → expects success. Any regression fails CI.

5. Tenant Isolation Guarantees & Continuous Verification

#	Guarantee	Mechanism	Automated check (P1-T20/P1-T21)
1	A request to <code>clienta...</code> can only ever open DB <code>clienta</code>	<code>db_filter=^%d\$</code> , <code>list_db=False</code>	Probe cross-subdomain data reads
2	Sessions never cross databases	Odoo DB-scoped sessions	Login A → visit B → must see login page
3	No shared tables between tenants	Database-per-tenant	Schema audit: no cross-DB objects
4	Attachments physically separated	Per-DB filestore dirs	Path audit in gate reviews
5	No tenant-reachable cross-DB code path	Two-layer separation; sync is platform-initiated only	Code review rule + grep audit for <code>db_connect</code> /cursor abuse in tenant-installed modules
6	Platform models absent from tenant DBs	Installation matrix (master plan §2.5)	Provisioning test asserts module set
7	Portal users see only their own documents	<code>ir.rule</code> + IDOR tests	P6-T02 IDOR suite

The isolation suite is a permanent CI citizen. It was designed in P1-T20, expands at every phase gate, and any failure is a release blocker of the highest severity — treated as a security incident, not a bug.

6. Authentication & Session Security

Control	Implementation	Task
Brute-force lockout	OCA <code>auth_brute_force</code> (port to 19.0 if needed) + Nginx <code>limit_req</code> on <code>/web/login</code> — two independent layers	P1-T19 / P1-T03
Session timeout	OCA <code>auth_session_timeout</code> , configurable per tenant	P1-T19
Cookie flags	<code>Secure</code> , <code>HttpOnly</code> , <code>SameSite=Lax</code> — verified, not assumed	P1-T19

CSRF	Odoo built-in tokens; login template overrides touch the template ONLY, never controller logic	P1-T14
Auth audit	<code>ncollection.auth.log</code> : success/failure/logout/reset with IP, UA, DB	P1-T19
Password policy	Odoo password policy params; provisioning forces reset of the initial admin password	P2-T01
Password reset	Odoo's flow verified: time-limited, single-use tokens; reset emails branded	P1-T19 / P1-T18
Debug/dev mode	Blocked for non-Owner; <code>?debug=</code> param neutralized	P1-T11
2FA (TOTP)	OCA evaluation → enforceable per tenant policy	P10-T04
SSO (enterprise tenants)	External IdP (Keycloak/Auth0) — deliberately deferred; adds infra before it pays	P10-T04
Service accounts	Config-sync + satellite accounts: unique, scoped, non-interactive login where supportable, rotated (§7)	P2-T03+

7. Secrets Management

Lifecycle by stage — the rule is *never in git, ever* (enforced by `.gitignore` + CI secret scanning):

Stage	Mechanism
Now (P1-T02)	<code>.env</code> file, mode 600, owned by deploy user; <code>.env.example</code> documents every variable without values
Production (P2-T08)	Docker secrets / root-owned env files; distinct secrets per environment (staging keys ≠ prod keys)
CI	GitHub Actions encrypted secrets; least scope; no secret ever echoed to logs
Enterprise (P10-T04)	Vault-class store with rotation and access audit

Inventory & rotation (kept in `docs/RUNBOOK_SECURITY.md` — names only, never values): DB password · Odoo `admin_passwd` (master password — disabled/unset in production once provisioning no longer needs the DB-manager API) · config-sync service account · SMTP creds · B2/S3 keys · pgBackRest cipher key · Stripe/PayTabs keys + webhook signing secrets · LLM API keys · FCM server key. Rotation: quarterly for high-power secrets, immediately on any suspected exposure, and on any team change.

Blast-radius rule: no satellite container gets a secret outside its job — the AI gateway holds LLM keys but no DB credentials; the backup agent holds B2 write keys but no payment secrets ([ARCHITECTURE_DATA_PLATFORM.md §10.4](#)).

8. Data Protection: Encryption & Backups

Data state	Protection
------------	------------

In transit (external)	TLS 1.2+ everywhere; HSTS; SSL Labs grade A verified at P3-T12
In transit (internal)	Single-host Docker network now; TLS on PG replication links from Stage 2; region links via WireGuard at Stage 4
At rest (DB/filestore)	Encrypted VPS volumes (provider snapshots + LUKS where offered); field-level <code>pgcrypto</code> reserved for narrowly-scoped secrets (e.g. stored API tokens) — full at-rest DB encryption relies on volume + backup encryption
Backups & WAL	pgBackRest repo cipher (AES-256) + encrypted tenant dumps; keys held outside the backup provider
Deletion	Tenant offboarding = DB drop + filestore purge + backup expiry per retention + certified deletion log (P10-T07)

Backups are treated as **a copy of the crown jewels stored off-site** — access to the B2 bucket is scoped to the backup agent's write-mostly key; restore-capable keys live in the secrets store, not on the server.

9. Compliance: UAE PDPL & Beyond

The UAE **Personal Data Protection Law (Federal Decree-Law No. 45 of 2021)** is the primary regime (GDPR-equivalent in structure). The platform is a **processor** for tenants' data and a **controller** for platform/billing data.

PDPL obligation	Platform answer	Task
Lawful processing & consent	Checkout consent capture; consent registry	P2-T16 / P10-T07
Data subject rights (access/portability)	Tenant data export (full DB dump + filestore in open formats)	P10-T07
Right to erasure	Verified offboarding workflow with deletion certificate	P10-T07
Security of processing	This entire document; assessment evidence at P3-T12	
Breach notification	Incident runbook includes notification decision tree + 72h clock	P3-T13 (<code>RUNBOOK_INCIDENTS.md</code>)
Data residency (market expectation)	Region-aware placement — UAE tenants' data, filestore, and backups stay in-region	P10-T05
Records of processing	Documented data inventory per module	P10-T07

Also tracked: **UAE e-invoicing** readiness (QR groundwork at P3-T09; full compliance when mandated), LGPL v3 obligations for Odoo rebranding (`docs/LEGAL.md` , P1-T13), PCI-DSS via **tokenization only** — card data never touches NCollection servers (P2-T13/P6-T01).

10. Security Operations

Practice	Cadence	Owner
OS security patches (unattended-upgrades)	Continuous	automated
Odoo/OCA/pip dependency review & bump (pinned via <code>repos.yml</code>)	Monthly + on CVE alerts	DEV-1
<code>pip-audit</code> + <code>trivy</code> image scan	Every CI run	automated (P1-T05)
Secret scan on commits	Every CI run	automated
Auth-log & audit-trail anomaly review	Weekly ops review	DEV-1
Access review (GitHub, servers, dashboards)	Quarterly + on team change	Omar
Restore drills	Per ARCHITECTURE_DATA_PLATFORM.md §5.3	DEV-1
Security assessment	Pre-launch (P3-T12), external pen test (P10-T04), then annually	team

Incident response (`docs/RUNBOOK_INCIDENTS.md` , written at P3-T13): severity ladder (SEV1 = suspected cross-tenant breach or data loss) → containment steps per scenario (credential compromise, tenant breach, ransomware, payment fraud) → communication templates (internal, affected tenants, PDPL notification) → post-mortem within 5 working days, blameless, with actions ticketed.

Non-negotiable SEV1 rule: on any suspicion of cross-tenant data exposure — freeze deployments, snapshot evidence, verify isolation suite against production, and treat every finding as real until proven otherwise.

11. Platform-Layer Specific Risks

Controls for the surfaces vanilla Odoo doesn't have (§2.3):

Surface	Risks	Controls
Provisioning engine (P2-T01/02)	DB-name injection, tenant overwrite, resource exhaustion, half-provisioned zombies	Strict name sanitization + reserved-word list + collision check; idempotent steps; rollback on failure; runner isolation (own container, direct DB conn, resource limits); provisioning quota per hour
Config sync channel (P2-T03)	Its service account is a skeleton key for tenant configs	Dedicated account, scoped to <code>ncollection.workspace.config</code> writes; credentials in secrets store; every sync logged; nightly reconciliation detects tampering/drift
Public checkout (P2-T16)	Mass fake signups → DB-creation DoS; subdomain squatting; injection via company fields	reCAPTCHA; rate limits; email verification before provisioning fires; reserved/offensive subdomain lists; full input validation; trial quotas per IP/email domain

License enforcement (P1-T10)	Bypass = revenue loss + precedent of broken guarantees	Continuous E2E probes (Ring 2 verification, §4); enforcement code changes require 2 reviewers
Payment webhooks (P2-T13/P6-T01)	Forged "paid" events, replay	Signature verification (Stripe signing secret / HMAC), timestamp tolerance, idempotency keys, amounts revalidated against the invoice — never trusted from the payload
Admin DB	Single point whose compromise = everything	Reachable only via <code>admin.</code> subdomain; platform-admin group + strong auth (2FA earliest adopter); most-audited DB; separate backup encryption scope

12. Per-Phase Security Checklists

Phase 1 gate (P1-T21)

- ☐ Isolation suite green (all 7 guarantees, §5)
- ☐ License enforcement probes green — URL **and** RPC (§4)
- ☐ Role matrix audited: no `implied_ids` escalation beyond `docs/ROLE_MATRIX.md`
- ☐ `list_db=False`; `/web/database/*` blocked at edge AND app; debug blocked
- ☐ Brute-force lockout + session timeout demonstrated; auth log populated
- ☐ Cookie flags verified; CSRF untouched by template overrides
- ☐ Security headers present (spot-check with curl); TLS redirect enforced
- ☐ No secrets in git (scan run); `.env.example` complete

Phase 2 gate (P2-T18)

- ☐ Provisioning: name sanitization tests, rollback proof, quota + runner isolation verified
- ☐ Config-sync account scoped + logged; reconciliation cron proven to heal drift
- ☐ Checkout abuse controls live (reCAPTCHA, email verification, rate + trial quotas)
- ☐ Stripe webhook signature + idempotency + amount revalidation tested with forged payloads
- ☐ PITR restore rehearsed; backup encryption verified; WAL-lag alert fires
- ☐ Server hardening: port scan clean, SSH password auth rejected, fail2ban active

Phase 3 / go-live gate (P3-T12 → P3-T13)

- ☐ Full assessment per P3-T12 (OWASP probing of login/checkout/portal) — zero unresolved critical/high
- ☐ SSL Labs grade A; headers audit; dependency scan review
- ☐ Incident runbook + on-call agreed; breach notification tree in place
- ☐ Restore drill ON PRODUCTION infrastructure completed
- ☐ Rate limits load-tested; monitoring alerts verified end-to-end

Later phases — each gate re-runs the cumulative checklist above, plus: portal IDOR suite (P6), JWT/device auth review (P7), OAuth2 scope audit + webhook HMAC tests (P8), marketplace code-signing chain (P9), pen test + failover-under-attack drill (P10).

Document End Owned jointly by DEV-1 (Layers 1, 4, 5) and DEV-2 (Layers 2, 3). Any PR that weakens a control in this document must say so explicitly in its description and obtain sign-off from both owners.